

Task 1

Michael König <m.koenig@kit.edu>

Paul Seehofer <paul.seehofer@kit.edu>

Deadline for submission: 2026-05-06T14:45

In this task, we take first steps in flow programming.

Task 1.1: Proactive and Reactive Flow Programming

- **Filename:** This task has two separate solutions (see details below)
 - ppsdn26_uxxxx_task11_proactive.py
 - ppsdn26_uxxxx_task11_reactive.py
- **Scenario file:** forwarding.yaml
- **Automatic evaluation:** YES
- **Topology:** See Figure 1 (One switch, two traffic generating networks)
- **Traffic:** Network N1 and N2 send IP flows to each other with a source IP address that belongs to their respective IP range (i.e., 11.0.0.0/8 for N1). The destination IP address is within the IP address range of the remote network (i.e., 22.0.0.0/8 for N1).
- **Assignment:** Program switch S1 to forward traffic to the correct networks. Implement two controller apps, one using proactive and the other using reactive flow programming.
- **Hints/Conditions**
 - All flow rules should exclusively use IP for matching!

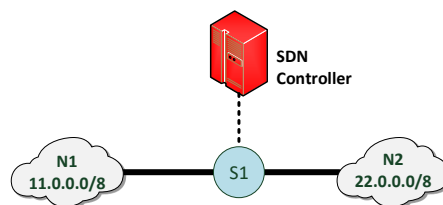


Figure 1: Scenario Task 1.1

Task 1.2: Learning Switch

- **Filename:** This task has three separate solutions (see details below)
 - ppsdn26_uxxxx_task12_single.py
 - ppsdn26_uxxxx_task12_double.py
 - ppsdn26_uxxxx_task12_twotable.py
- **Scenario file:** learning.yaml
- **Automatic evaluation:** NO
- **Topology:** See Figure 2 (1 switch (S1) and 4 connected hosts (H1, H2, H3, H4))
- **Traffic:** This task has no automatic traffic generation. You have to use the pingall button and suitable debug output from your application to test your solution!
- **Assignment:** For this task, you have to develop a learning switch application in three different ways. The basic functionality of a learning switch is to „learn“ the location of the hosts in the network. As soon as a host – let’s say H1 – starts a new communication attempt, the packets of this (currently unmatched) flow are inspected by the controller. This way, your application is able to learn the location of the new host (H1) by inspecting the source address and the ingress port. Based on this information, the application can manage a forwarding information base and set up forwarding rules based on Ethernet addresses. There are several (conceptually different!) ways to implement this functionality:
 - ppsdn26_uxxxx_task12_single.py: Only one flow table and only one distinct match field for the forwarding rules (layer 2 destination address)
 - ppsdn26_uxxxx_task12_double.py: Only one flow table but with two different match fields
 - ppsdn26_uxxxx_task12_twotable.py: Using two flow tables, one for learning and one for forwarding
- **Hints/Conditions**
 - Only reactive approaches are allowed (otherwise it’s not “learning”)
 - The learning switch is explained in more detail in the telematics lecture.
 - Use OpenFlow instructions and ryu’s OFPInstructionGotoTable instruction when programming multiple flow tables.

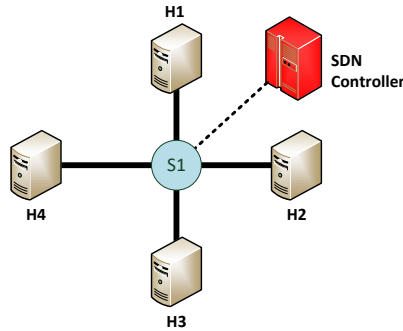


Figure 2: Scenario Task 1.2

Task 1.3: Priority-Based Filtering

- **Filename:** ppsdn26_uxxxx_task13.py
- **Scenario file:** priority_filter.yaml
- **Automatic evaluation:** YES
- **Topology:** See Figure 3 (1 switch, 1 sender and 2 receiver networks)
- **Traffic:** Network N1 sends IP packets with IP source addresses from the subnet 11.0.0.0/8 to network N2 and N3 with destination addresses in the range 22.0.0.0/8 and 33.0.0.0/8 respectively. However, due to a misconfiguration packets from the non-public IP subnet 10.0.0.0/8 leak into the public network.
- **Assignment:** Forward the correct traffic to N2 and N3, but prevent switch S1 from forwarding any IP packets with non-public IP source addresses (i.e., from 10.0.0.0/8), even when these are destined to N2 or N3.
- **Hints/Conditions**
 - All flow rules should exclusively use IP for matching!
 - Use priority 0 for default flows.
 - Arrange the priorities of your flowrules carefully.

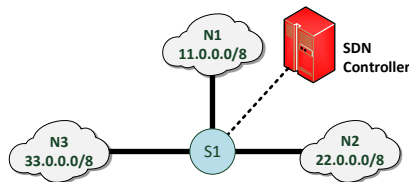


Figure 3: Scenario Task 1.3

Task 1.4: Timeout-Based Mirroring

- **Filename:** ppsdn26_uxxxx_task14.py
- **Scenario file:** timed_mirror.yaml
- **Automatic evaluation:** YES
- **Topology:** See Figure 4 (1 switch, 1 sender and 3 receiver networks)
- **Traffic:** Network N1 sends IP packets to network N2 and N3 with destination addresses in the range 22.0.0.0/8 and 33.0.0.0/8 respectively. However, multiple complaints indicate that N1 has been the source of suspicious traffic.
- **Assignment:** When an IP packet arrives at switch S1 that is not destined to N2 or N3 the switch shall drop all traffic directed at the /8 subnet containing the IP address of that packet (e.g., subnet 10.0.0.0/8 for destination IP 10.1.1.1). However, to gain a better understanding of the situation, the suspicious traffic shall be forwarded to the network N4 for analysis first. Program S1 to forward all traffic to N2 and N3 as usual. Examine all packets not destined to N2 or N3 and determine their IP destination address and their /8 subnet. Program flow rules to redirect all traffic to those subnets to N4 for 4 seconds and drop all packets of those flows afterwards.
- **Hints/Conditions**
 - All flow rules should exclusively use IP for matching!
 - Use the right kind of timeouts for the traffic redirection to N4.
 - Choose appropriate priorities for the drop- and redirection-flows.

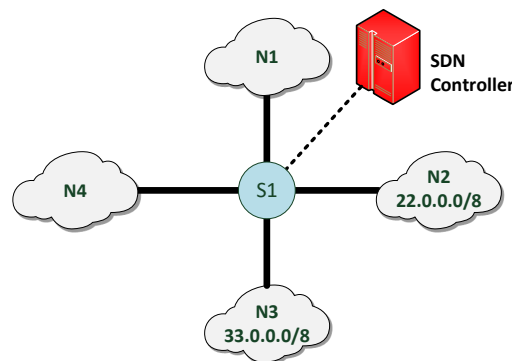


Figure 4: Scenario Task 1.4